



机器学习与 Python 实践

第 3 次作业

基于汽油类型的汽车油耗预测模型

姓名：

专业： 强基班

学号：

教师：

学院： 信息科学与工程学院

2026 年 5 月 25 日

目录

1	前言及问题概述	2
2	相关基础知识及方法	2
2.1	相关原理	2
2.2	数据描述	3
2.3	算法实现	5
3	实验结果与可扩展内容	6
3.1	实验结果	6
3.2	参数调优及可扩展内容	10
4	讨论与结论	10
4.1	结果讨论	10
4.2	结论总结	11
5	心得体会与思考	11
5.1	个人心得	11
5.2	启发与思考	11
6	附录	12
	算法代码及其注释	12
	关键运行结果汇总	24

1 前言及问题概述

汽车油耗是车辆使用成本和能源效率的重要指标。对于同一辆车，如果行驶路线、驾驶习惯和天气条件较为接近，那么汽油类型、平均速度、行驶距离、车内外温度、是否开空调以及是否下雨等因素都可能影响单位百公里油耗。题目给出的案例中，James 长期驾驶同一辆车，并在加油站交替使用 E10 和 SP98 两类汽油，其中 E10 单价较低，但经验上使用 E10 时车辆耗油量更高。因此，本次实验的核心任务是根据给定行驶记录，建立模型预测汽车油耗 `consume`，并分析汽油类型在预测中的作用。

本实验使用 `measurements.xlsx` 数据集，共 388 条样本，包含 11 个输入特征和 1 个目标特征。目标变量为 `consume`，表示每百公里消耗燃油升数，单位为 L/100km。输入特征包括行驶距离、平均速度、车内温度、车外温度、特殊事件、汽油类型、空调、雨天、晴天、补充汽油升数和补充汽油类型。实验先进行数据预览、缺失值分析和探索性可视化，再构造若干补充特征，最后比较仅使用汽油类型的基线模型、线性回归、岭回归、随机森林回归和梯度提升回归的预测效果。

需要特别说明的是，本题虽然强调“根据汽油类型预测消耗量”，但真实油耗并不只由汽油类型决定。仅使用 `gas_type` 建模能够检验汽油类型自身的解释能力，而使用全部行驶和环境特征建模则更接近实际预测场景。两类结果结合起来，可以更清楚地说明汽油类型与油耗之间的关系。

2 相关基础知识及方法

2.1 相关原理

本任务属于监督学习中的回归问题。设第 i 条行驶记录的输入特征为 $x^{(i)}$ ，目标油耗为 $y^{(i)}$ ，回归模型的目标是学习函数 $f(x)$ ，使预测值

$$\hat{y}^{(i)} = f(x^{(i)}) \quad (1)$$

尽可能接近真实油耗。模型效果通过测试集上的误差指标评价。

线性回归假设目标变量与特征之间近似满足线性关系：

$$\hat{y} = w_0 + w_1x_1 + \cdots + w_dx_d. \quad (2)$$

该模型结构清晰、可解释性强，但对非线性关系和特征交互的表达能力有限。岭回归在线性回归基础上加入 L_2 正则化项，目标函数为

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2 + \lambda \|\mathbf{w}\|_2^2, \quad (3)$$

能够在一定程度上缓解多重共线性和过拟合。

随机森林回归属于集成学习方法。它通过多个决策树在不同样本和特征子集上学习规律，并取多棵树预测结果的平均值：

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T f_t(x). \quad (4)$$

与线性模型相比，随机森林能够自动捕捉非线性关系和变量交互，因此适合处理行驶距离、速度、天气和空调等因素共同影响油耗的表格型数据。梯度提升回归同样基于决策树，但采用逐步拟合残差的方式不断提升模型表现。

本实验使用 RMSE、MAE、 R^2 和 NRMSE 四个指标评价模型。RMSE 定义为

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2}, \quad (5)$$

MAE 定义为

$$MAE = \frac{1}{n} \sum_{i=1}^n |\hat{y}^{(i)} - y^{(i)}|, \quad (6)$$

R^2 定义为

$$R^2 = 1 - \frac{\sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2}{\sum_{i=1}^n (y^{(i)} - \bar{y})^2}. \quad (7)$$

其中 RMSE、MAE 和 NRMSE 越小越好， R^2 越接近 1 表示模型解释能力越强。

2.2 数据描述

数据集文件为 `measurements.xlsx`，读取后共 388 行、12 列，其中 `consume` 为目标变量，其余 11 列作为候选输入特征。主要字段说明见表 1。

表 1: 汽车油耗数据集字段说明

字段	类型	含义
<code>distance</code>	连续特征	行车距离，单位 km
<code>consume</code>	目标变量	消耗量，单位 L/100km
<code>speed</code>	连续特征	平均速度，单位 km/h
<code>temp_inside</code>	连续特征	车内温度，单位 °C
<code>temp_outside</code>	连续特征	车外温度，单位 °C
<code>specials</code>	类别特征	是否发生特殊事件，如雨、阳光、空调等
<code>gas_type</code>	类别特征	使用汽油类型，E10 或 SP98
<code>AC</code>	二值特征	是否开启空调
<code>rain</code>	二值特征	是否下雨
<code>sun</code>	二值特征	是否阳光明媚

字段	类型	含义
refill_liters	连续特征	补充汽油升数
refill_gas	类别特征	补充汽油类型

数据中 distance、consume、speed、temp_outside、gas_type、AC、rain 和 sun 均无缺失值。temp_inside 有 12 个缺失值，specials 有 295 个缺失值，refill_liters 和 refill_gas 各有 375 个缺失值。由于补油字段缺失数量很高，缺失本身也包含“本次记录没有补油信息”的含义，因此实验没有直接删除这些样本，而是将类别缺失填充为 none，数值缺失使用中位数填充，并额外构造 has_refill 和 has_special 两个二值特征。

目标变量 consume 的均值为 4.9124，标准差为 1.0332，最小值为 3.3，最大值为 12.2。不同汽油类型下的油耗统计见表 2。

表 2: 不同汽油类型下的油耗统计

汽油类型	样本数	均值	中位数	标准差	最小值	最大值
E10	160	4.9313	4.8000	0.9010	3.7	10.8
SP98	228	4.8991	4.7000	1.1184	3.3	12.2

从表中可以看到，E10 的平均油耗略高于 SP98，但差异仅约 0.032 L/100km，远小于样本自身波动。因此，汽油类型可能提供一定信息，但很难单独解释油耗变化。

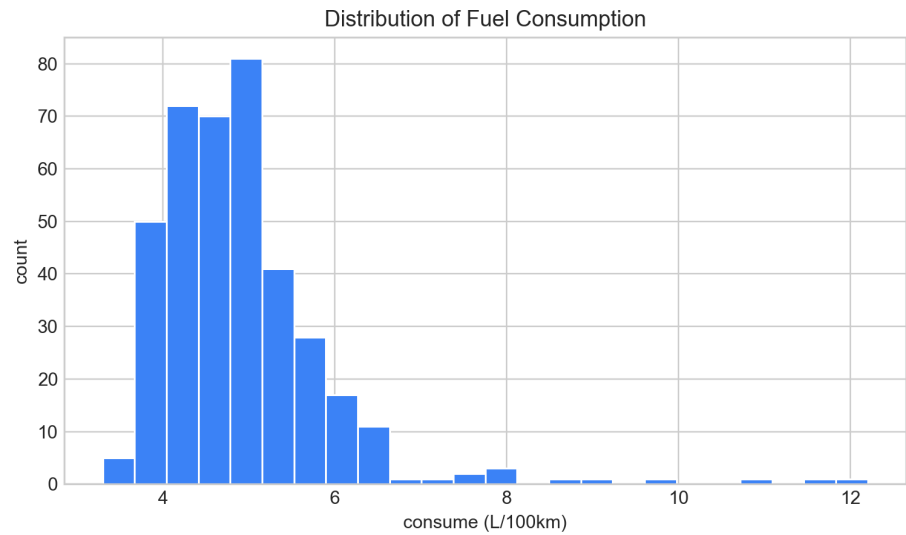


图 1: 目标变量 consume 分布

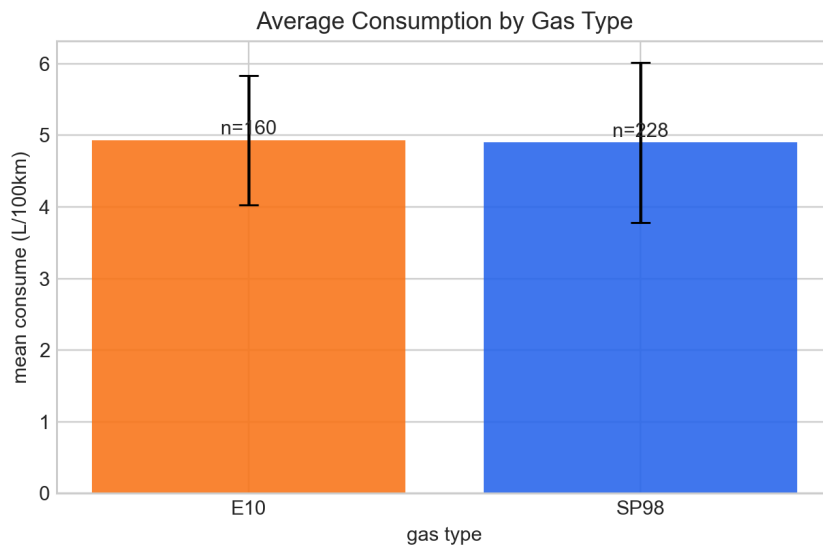


图 2: 不同汽油类型的平均油耗

2.3 算法实现

实验程序使用 `pandas` 读取 Excel 数据, 使用 `scikit-learn` 完成预处理、模型训练和模型评价。主要流程如下:

1. 读取 `measurements.xlsx`, 确认数据规模、字段类型、缺失值和描述性统计。
2. 对 `gas_type` 分组统计平均油耗, 并绘制目标变量分布、汽油类型油耗柱状图、速度与油耗散点图和相关性热力图。
3. 构造 `has_special`、`has_refill`、`temp_diff_inside_outside`、`distance_per_speed`、`speed_temp_interaction` 等补充特征。
4. 将数据按 80%/20% 随机划分为训练集和测试集, 其中训练集 310 条, 测试集 78 条。
5. 对数值特征使用中位数填充并进行标准化, 对类别特征使用常量 `none` 填充后进行 One-Hot 编码。
6. 分别训练 `GasType_LinearRegression`、`Full_LinearRegression`、`Full_Ridge`、`Full_RandomForest` 和 `Full_GradientBoosting` 五个模型。
7. 在测试集上计算 RMSE、MAE、 R^2 和 NRMSE, 同时使用 5 折交叉验证估计模型稳定性。
8. 保存最佳模型的真实值与预测值散点图、预测曲线以及随机森林特征重要性图。

完整 Python 程序见附录。

3 实验结果与可扩展内容

3.1 实验结果

图 3 展示了平均速度和油耗之间的关系。总体上，速度较低时油耗更容易偏高，这符合车辆低速行驶、频繁启停时燃油效率较低的经验。两类汽油样本在散点图中存在较多重叠，也说明汽油类型并不是唯一主导因素。

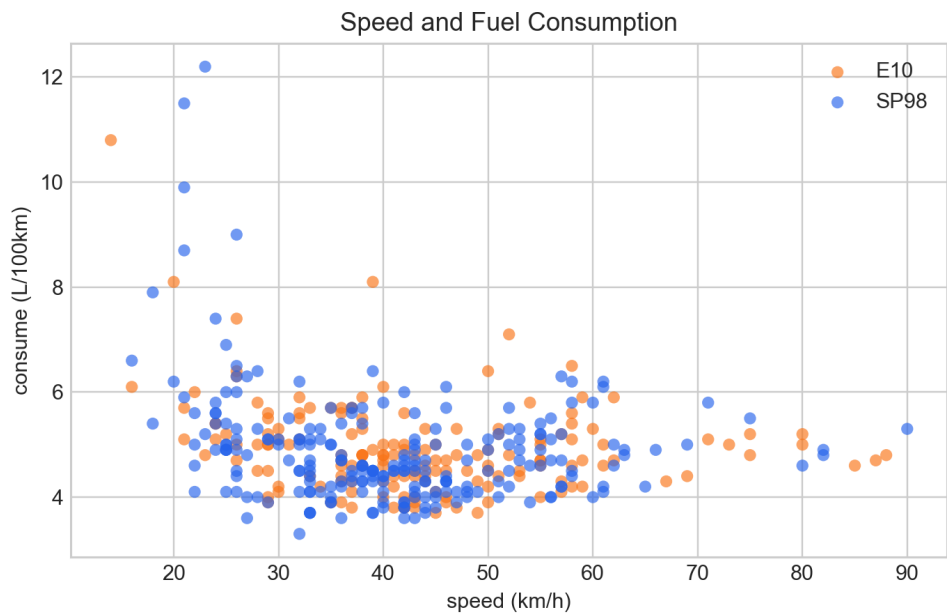


图 3: 平均速度与油耗的关系

数值特征相关性热力图见图 4。与目标变量 `consume` 关系较明显的是 `distance`、`speed`、`rain` 和 `AC` 等字段。其中行驶距离与油耗呈负相关，说明短距离行驶更容易出现较高单位油耗；平均速度与油耗也呈负相关，反映出低速或拥堵环境下油耗更高。

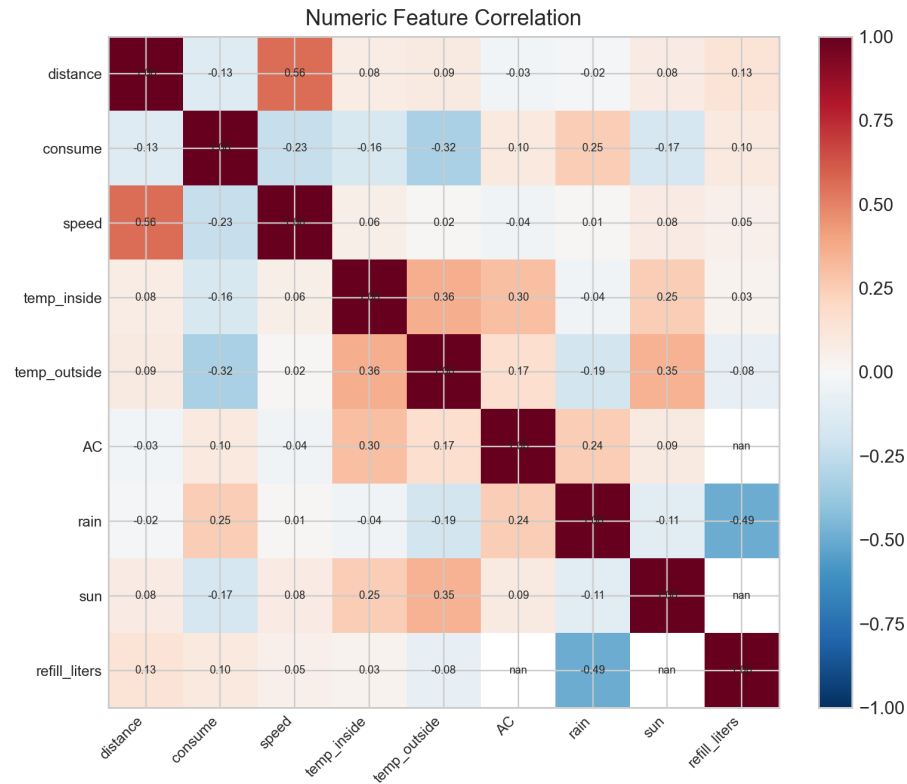


图 4: 数值特征相关性热力图

不同模型在测试集上的结果见表 3。仅使用汽油类型的线性回归模型 GasType_LinearRegression 测试集 R^2 仅为 0.0023，几乎不能解释油耗波动。这与前文分组统计一致，说明 E10 与 SP98 的平均油耗差异较小，单独使用汽油类型预测不足以得到可靠模型。

表 3: 模型测试集评价结果

模型	RMSE	MAE	R^2	NRMSE
Full_RandomForest	0.6567	0.4285	0.5247	0.1042
Full_GradientBoosting	0.8041	0.4770	0.2874	0.1276
Full_LinearRegression	0.8513	0.6271	0.2012	0.1351
Full_Ridge	0.8558	0.6302	0.1928	0.1358
GasType_LinearRegression	0.9514	0.6396	0.0023	0.1510

加入全部行驶和环境特征后，模型效果明显提升。其中随机森林表现最好，测试集 RMSE 为 0.6567，MAE 为 0.4285， R^2 为 0.5247，说明模型能够解释约 52.47% 的测试集油耗波动。梯度提升模型次之，线性回归和岭回归低于树模型，说明本数据中存在一定非线性关系和特征交互。

表 4 给出了 5 折交叉验证平均结果。随机森林的交叉验证 RMSE 为 0.6836，平均

R^2 为 0.4964，与测试集结果较接近，说明模型表现具有一定稳定性。

表 4: 模型 5 折交叉验证结果

模型	CV RMSE 均值	CV MAE 均值	CV R^2 均值
Full_RandomForest	0.6836	0.4525	0.4964
Full_GradientBoosting	0.7244	0.4898	0.4308
Full_LinearRegression	0.9942	0.6494	-0.0384
Full_Ridge	0.9681	0.6473	0.0087
GasType_LinearRegression	1.0153	0.6756	-0.0485

最佳模型的预测效果见图 5 和图 6。从散点图看，随机森林的多数预测点靠近理想预测直线，但对于个别高油耗样本仍存在低估现象。这与数据集中高油耗样本数量较少有关，模型更容易学习到常见油耗范围内的规律。

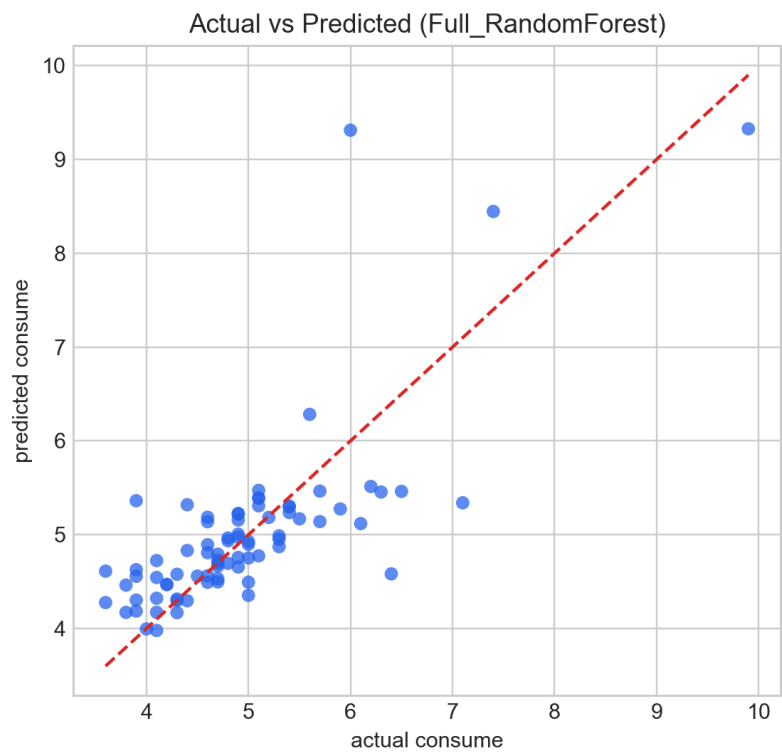


图 5: 最佳模型真实值与预测值散点图

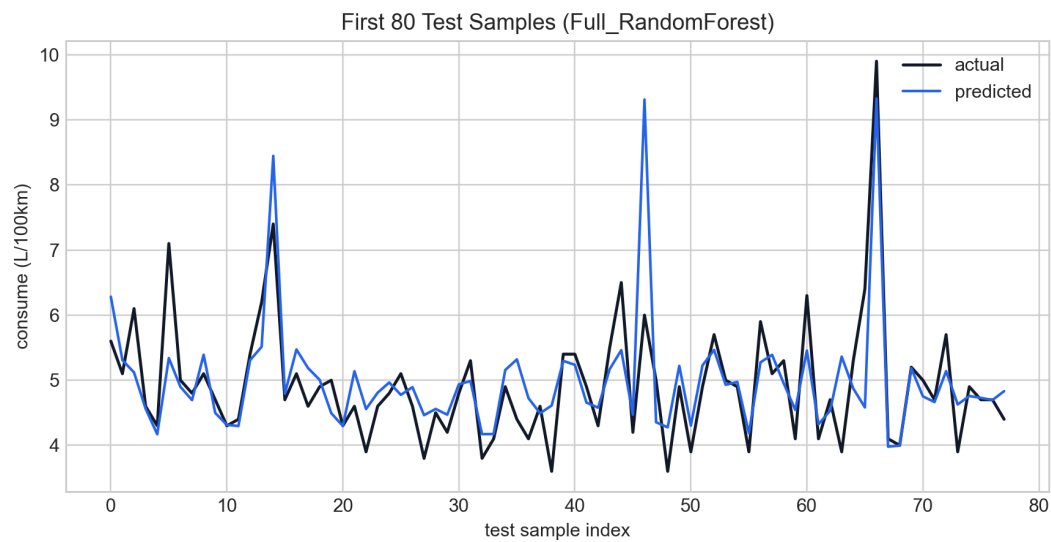


图 6: 最佳模型测试集前 80 个样本预测曲线

随机森林特征重要性见图 7。结果显示，`distance` 的重要性最高，为 0.5014；其次是 `distance_per_speed`、`speed_temp_interaction`、`speed` 和 `temp_outside`。与之相比，`gas_type_SP98` 和 `gas_type_E10` 的重要性分别约为 0.0021 和 0.0020，说明在当前数据和模型中，汽油类型对油耗预测的贡献较小，行驶状态和环境因素更关键。

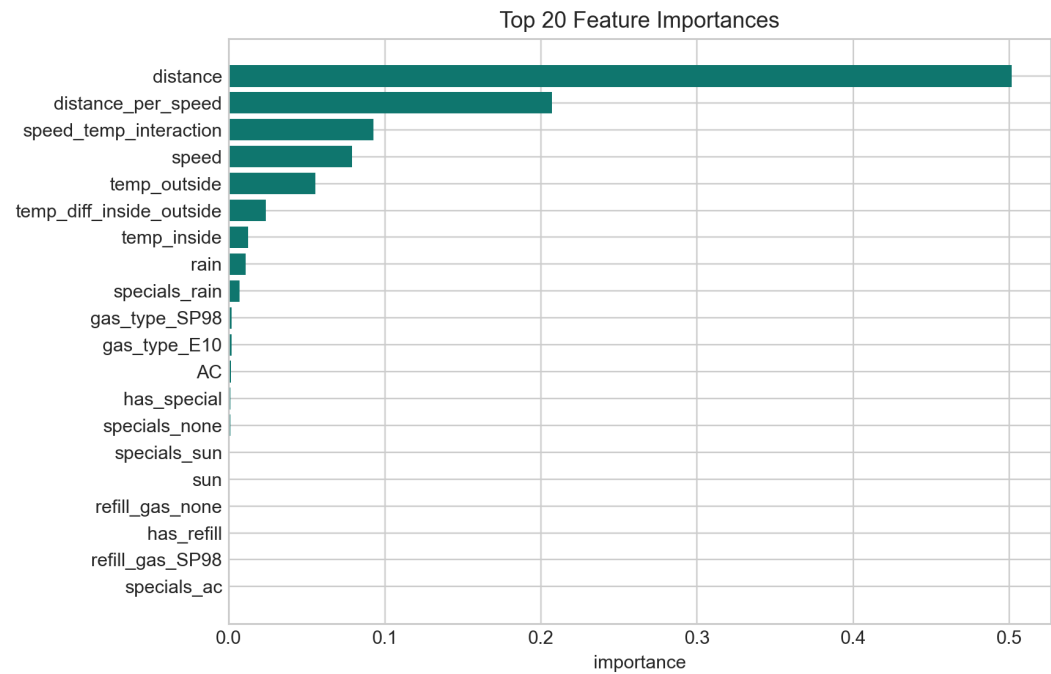


图 7: 随机森林前 20 个重要特征

3.2 参数调优及可扩展内容

本次实验使用固定参数完成模型比较。随机森林设置为 500 棵树，最小叶节点样本数为 3，以降低小样本数据上的过拟合风险。梯度提升模型设置 180 棵树、学习率为 0.04、树深为 3。后续若继续优化，可以从以下方向扩展：

- 使用网格搜索或随机搜索调节随机森林的 `max_depth`、`min_samples_leaf` 和 `max_features`。
- 对梯度提升模型进一步搜索 `n_estimators`、`learning_rate` 和 `subsample`，在偏差和方差之间取得更好平衡。
- 加入更贴近驾驶场景的特征，例如短距离行驶标记、低速行驶标记、雨天且开空调交互、温差分箱等。
- 若后续有时间顺序或路线信息，可以构造滚动平均油耗、上一段行程油耗等序列特征。
- 增加样本数量，特别是高油耗和不同汽油类型在相近路况下的样本，以更可靠地评估汽油类型影响。

4 讨论与结论

4.1 结果讨论

通过实验可以得到以下结论。第一，仅使用汽油类型预测油耗的效果很弱。E10 平均油耗为 4.9313 L/100km，SP98 平均油耗为 4.8991 L/100km，差异很小；对应的单特征线性回归测试集 R^2 仅为 0.0023，说明汽油类型本身不能充分解释油耗变化。

第二，加入行驶距离、速度、温度、天气和空调等特征后，预测效果显著提升。随机森林测试集 R^2 达到 0.5247，RMSE 降至 0.6567，说明车辆油耗更像是多个因素共同作用的结果，而不是单一汽油类型的简单函数。

第三，树模型优于线性模型。线性回归和岭回归的 R^2 约为 0.20，而随机森林超过 0.52，说明数据中存在非线性关系。例如短距离、低速度、温度和天气条件会相互影响单位油耗，简单线性模型难以充分表达这些关系。

第四，特征重要性显示行驶距离和速度相关特征最关键。`distance`、`distance_per_speed`、`speed_temp_interaction` 和 `speed` 位居前列，说明驾驶场景对油耗的影响大于汽油类型。该结果也提醒我们，在机器学习建模中应避免只关注题目中显眼的变量，而要结合业务含义综合分析特征。

4.2 结论总结

本次实验完成了基于汽车行驶记录的油耗预测任务。实验从 Excel 数据读取开始，依次完成数据预览、缺失值处理、探索性分析、特征构造、数值标准化、类别编码、模型训练和模型评价。结果表明，汽油类型 E10 与 SP98 在平均油耗上存在轻微差异，但单独使用汽油类型难以建立有效预测模型；加入行驶距离、平均速度、车内外温度、天气和空调等特征后，随机森林取得最佳预测效果，测试集 RMSE 为 0.6567，MAE 为 0.4285， R^2 为 0.5247。

综合来看，汽车油耗预测需要同时考虑燃油类型、行驶状态和外部环境。对于本数据集而言，路线距离和平均速度是更重要的解释因素，汽油类型只提供较弱的辅助信息。后续若要更准确判断 E10 与 SP98 的实际经济性，需要在相同路线、相近速度和相似天气条件下收集更多成对样本，再结合燃油价格和预测油耗计算综合使用成本。

5 心得体会与思考

5.1 个人心得

通过本次实验，我进一步体会到机器学习任务中“理解数据含义”比直接套用模型更重要。题目表面上强调汽油类型，但真正分析后发现，油耗与距离、速度、温度和天气等变量关系更紧密。如果只根据题目描述直接使用 `gas_type` 建模，得到的模型几乎没有预测能力；只有把数据集中的其他变量一起纳入，才能更接近真实驾驶场景。

同时，本次实验也让我认识到缺失值并不一定只是需要删除或填充的异常。在 `refill_liters` 和 `refill_gas` 字段中，缺失往往表示该条记录没有补油信息，因此我额外构造了 `has_refill` 特征保留这一信息。类似地，`specials` 缺失也可解释为没有记录特殊事件。这样的处理比简单删除样本更合理。

5.2 启发与思考

本实验说明，建模结果必须结合业务背景解释。E10 售价低于 SP98，但如果 E10 导致油耗升高，那么最终使用成本不一定更低。不过本数据集中两类汽油平均油耗差异很小，且样本受到速度、距离和天气等因素影响较大，因此不能仅凭当前结果断言哪种汽油一定更经济。更严谨的做法是控制路线和驾驶条件，收集更多可比样本，再将“每百公里燃油费用”作为最终评价指标。

后续改进时，我希望尝试两类方向：一是引入更强的模型和交叉验证调参，提高预测稳定性；二是从业务问题出发，将油耗预测扩展为成本预测，例如分别计算 E10 和 SP98 的单位里程费用，从而为驾驶者选择汽油类型提供更直接的参考。

6 附录

算法代码及其注释

```
# -*- coding: utf-8 -*-
"""
报告 3: 基于汽车行驶记录的油耗预测实验。

任务目标:
1. 读取 measurements.xlsx 数据集;
2. 以 consume (L/100km) 为目标变量, 根据汽油类型及其他行驶/环境特征预测油耗;
3. 完成数据预处理、探索性分析、建模评价和结果可视化;
4. 输出报告所需的图表与关键结果文件。
"""

from __future__ import annotations

import argparse
import json
import warnings
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import KFold, cross_validate, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

warnings.filterwarnings("ignore")
```

```
TARGET_COL = "consume"
RANDOM_STATE = 42

def ensure_dir(path: Path) -> Path:
    path.mkdir(parents=True, exist_ok=True)
    return path

def make_one_hot_encoder() -> OneHotEncoder:
    try:
        return OneHotEncoder(handle_unknown="ignore", sparse_output=False)
    except TypeError:
        return OneHotEncoder(handle_unknown="ignore", sparse=False)

def load_data(data_path: Path) -> pd.DataFrame:
    if not data_path.exists():
        raise FileNotFoundError(f" 找不到数据文件: {data_path}")
    df = pd.read_excel(data_path)
    expected_cols = {
        "distance",
        "consume",
        "speed",
        "temp_inside",
        "temp_outside",
        "specials",
        "gas_type",
        "AC",
        "rain",
        "sun",
        "refill_liters",
        "refill_gas",
    }
    missing_cols = expected_cols - set(df.columns)
    if missing_cols:
        raise ValueError(f" 数据集缺少字段: {sorted(missing_cols)}")
    return df

def add_features(df: pd.DataFrame) -> pd.DataFrame:
```

```

data = df.copy()
data["has_special"] = data["specials"].notna().astype(int)
data["has_refill"] = data["refill_liters"].notna().astype(int)
data["temp_diff_inside_outside"] = data["temp_inside"] -
    ↪ data["temp_outside"]
data["distance_per_speed"] = data["distance"] / data["speed"].replace(0,
    ↪ np.nan)
data["speed_temp_interaction"] = data["speed"] * data["temp_outside"]
data["rain_ac"] = data["rain"] * data["AC"]
data["sun_ac"] = data["sun"] * data["AC"]
return data

def regression_metrics(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str,
    ↪ float]:
    rmse = mean_squared_error(y_true, y_pred, squared=False)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    nrmse = rmse / (float(np.max(y_true)) - float(np.min(y_true)))
    return {"RMSE": rmse, "MAE": mae, "R2": r2, "NRMSE": nrmse}

def build_preprocessor(X: pd.DataFrame) -> ColumnTransformer:
    categorical_features = [
        col
        for col in ["gas_type", "specials", "refill_gas"]
        if col in X.columns
    ]
    numeric_features = [col for col in X.columns if col not in
    ↪ categorical_features]

    numeric_pipeline = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
        ]
    )
    categorical_pipeline = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="constant",
    ↪ fill_value="none")),

```

```

        ("onehot", make_one_hot_encoder()),
    ]
)
return ColumnTransformer(
    transformers=[
        ("num", numeric_pipeline, numeric_features),
        ("cat", categorical_pipeline, categorical_features),
    ],
    remainder="drop",
)

def get_feature_names(preprocessor: ColumnTransformer) -> list[str]:
    names: list[str] = []
    for transformer_name, transformer, columns in preprocessor.transformers_:
        if transformer_name == "remainder" or transformer == "drop":
            continue
        if transformer_name == "cat":
            encoder = transformer.named_steps["onehot"]
            names.extend(encoder.get_feature_names_out(columns).tolist())
        else:
            names.extend(list(columns))
    return names

def save_overview(df: pd.DataFrame, out_dir: Path) -> None:
    ensure_dir(out_dir)
    df.head(10).to_csv(out_dir / "head_10.csv", index=False,
        ↪ encoding="utf-8-sig")
    df.dtypes.astype(str).rename("dtype").to_csv(
        out_dir / "dtypes.csv", encoding="utf-8-sig"
    )
    df.isna().sum().rename("missing_count").to_csv(
        out_dir / "missing_values.csv", encoding="utf-8-sig"
    )
    df.describe(include="all").to_csv(out_dir / "describe.csv",
        ↪ encoding="utf-8-sig")

gas_summary = (
    df.groupby("gas_type")[TARGET_COL]
    .agg(["count", "mean", "median", "std", "min", "max"])

```



```

        .sort_index()
    )
    gas_summary.to_csv(out_dir / "consume_by_gas_type.csv",
        ↪ encoding="utf-8-sig")

    binary_summary = (
        df[["AC", "rain", "sun", TARGET_COL]]
        .groupby(["AC", "rain", "sun"])
        .agg(["count", "mean"])
    )
    binary_summary.to_csv(out_dir / "consume_by_weather_ac.csv",
        ↪ encoding="utf-8-sig")

def plot_eda(df: pd.DataFrame, fig_dir: Path) -> None:
    ensure_dir(fig_dir)
    plt.style.use("seaborn-v0_8-whitegrid")

    fig, ax = plt.subplots(figsize=(7.0, 4.2))
    ax.hist(df[TARGET_COL], bins=24, color="#3B82F6", edgecolor="white")
    ax.set_title("Distribution of Fuel Consumption")
    ax.set_xlabel("consume (L/100km)")
    ax.set_ylabel("count")
    fig.tight_layout()
    fig.savefig(fig_dir / "target_consume_distribution.png", dpi=220)
    plt.close(fig)

    gas_stats = (
        df.groupby("gas_type")[TARGET_COL]
        .agg(["mean", "std", "count"])
        .reindex(["E10", "SP98"])
    )
    fig, ax = plt.subplots(figsize=(6.0, 4.0))
    ax.bar(
        gas_stats.index,
        gas_stats["mean"],
        yerr=gas_stats["std"],
        capsize=5,
        color=["#F97316", "#2563EB"],
        alpha=0.88,
    )

```

```

ax.set_title("Average Consumption by Gas Type")
ax.set_xlabel("gas type")
ax.set_ylabel("mean consume (L/100km)")
for i, (idx, row) in enumerate(gas_stats.iterrows()):
    ax.text(i, row["mean"] + 0.08, f"n={int(row['count'])}", ha="center")
fig.tight_layout()
fig.savefig(fig_dir / "consume_by_gas_type.png", dpi=220)
plt.close(fig)

fig, ax = plt.subplots(figsize=(6.4, 4.2))
for gas, color in [("E10", "#F97316"), ("SP98", "#2563EB")]:
    sub = df[df["gas_type"] == gas]
    ax.scatter(
        sub["speed"],
        sub[TARGET_COL],
        label=gas,
        alpha=0.65,
        s=32,
        color=color,
        edgecolor="none",
    )
ax.set_title("Speed and Fuel Consumption")
ax.set_xlabel("speed (km/h)")
ax.set_ylabel("consume (L/100km)")
ax.legend()
fig.tight_layout()
fig.savefig(fig_dir / "speed_vs_consume.png", dpi=220)
plt.close(fig)

numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
corr = df[numeric_cols].corr()
fig, ax = plt.subplots(figsize=(8.2, 6.2))
im = ax.imshow(corr, cmap="RdBu_r", vmin=-1, vmax=1)
ax.set_xticks(np.arange(len(corr.columns)))
ax.set_yticks(np.arange(len(corr.index)))
ax.set_xticklabels(corr.columns, rotation=45, ha="right", fontsize=8)
ax.set_yticklabels(corr.index, fontsize=8)
for i in range(len(corr.index)):
    for j in range(len(corr.columns)):
        ax.text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center",
            ↪ va="center", fontsize=6)

```

```

fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
ax.set_title("Numeric Feature Correlation")
fig.tight_layout()
fig.savefig(fig_dir / "correlation_heatmap.png", dpi=220)
plt.close(fig)

def evaluate_models(X: pd.DataFrame, y: pd.Series, out_dir: Path, fig_dir:
↪ Path) -> dict:
    ensure_dir(out_dir)
    ensure_dir(fig_dir)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=RANDOM_STATE
    )

    preprocessor = build_preprocessor(X_train)
    models = {
        "GasType_LinearRegression": Pipeline(
            steps=[
                (
                    "preprocess",
                    ColumnTransformer(
                        transformers=[
                            (
                                "cat",
                                Pipeline(
                                    steps=[
                                        (
                                            "imputer",
                                            SimpleImputer(
                                                strategy="constant",
                                                fill_value="none",
                                            ),
                                        ),
                                        ("onehot", make_one_hot_encoder()),
                                    ]
                                ),
                            ],
                        ),
                    ["gas_type"],
                )
            ]
        )
    }

```

```

        ),
    ),
    ("model", LinearRegression()),
]
),
"Full_LinearRegression": Pipeline(
    steps=[("preprocess", preprocessor), ("model",
        ↪ LinearRegression())]
),
"Full_Ridge": Pipeline(
    steps=[
        ("preprocess", build_preprocessor(X_train)),
        ("model", Ridge(alpha=1.0, random_state=RANDOM_STATE)),
    ]
),
"Full_RandomForest": Pipeline(
    steps=[
        ("preprocess", build_preprocessor(X_train)),
        (
            "model",
            RandomForestRegressor(
                n_estimators=500,
                max_depth=None,
                min_samples_leaf=3,
                random_state=RANDOM_STATE,
                n_jobs=-1,
            ),
        ),
    ]
),
"Full_GradientBoosting": Pipeline(
    steps=[
        ("preprocess", build_preprocessor(X_train)),
        (
            "model",
            GradientBoostingRegressor(
                n_estimators=180,
                learning_rate=0.04,
                max_depth=3,
                random_state=RANDOM_STATE,
            ),
        ),
    ]
),

```

```

        ),
    ]
),
}

cv = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
rows = []
predictions = {}
for name, model in models.items():
    scoring = {
        "neg_rmse": "neg_root_mean_squared_error",
        "neg_mae": "neg_mean_absolute_error",
        "r2": "r2",
    }
    cv_scores = cross_validate(model, X, y, cv=cv, scoring=scoring,
        ↪ n_jobs=-1)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    predictions[name] = y_pred
    metrics = regression_metrics(y_test, y_pred)
    rows.append(
        {
            "model": name,
            "test_RMSE": metrics["RMSE"],
            "test_MAE": metrics["MAE"],
            "test_R2": metrics["R2"],
            "test_NRMSE": metrics["NRMSE"],
            "cv_RMSE_mean": -cv_scores["test_neg_rmse"].mean(),
            "cv_RMSE_std": cv_scores["test_neg_rmse"].std(),
            "cv_MAE_mean": -cv_scores["test_neg_mae"].mean(),
            "cv_R2_mean": cv_scores["test_r2"].mean(),
        }
    )

result_df = pd.DataFrame(rows).sort_values("test_RMSE")
result_df.to_csv(out_dir / "model_comparison.csv", index=False,
    ↪ encoding="utf-8-sig")

best_name = result_df.iloc[0]["model"]
best_model = models[best_name]
best_pred = predictions[best_name]

```

```

prediction_df = pd.DataFrame({"actual": y_test, "predicted": best_pred})
prediction_df.to_csv(
    out_dir / "best_model_predictions.csv", index=False,
    ↪ encoding="utf-8-sig"
)

fig, ax = plt.subplots(figsize=(5.2, 5.0))
ax.scatter(y_test, best_pred, s=38, alpha=0.75, color="#2563EB",
    ↪ edgecolor="none")
low = min(float(y_test.min()), float(np.min(best_pred)))
high = max(float(y_test.max()), float(np.max(best_pred)))
ax.plot([low, high], [low, high], color="#DC2626", lw=1.5,
    ↪ linestyle="--")
ax.set_title(f"Actual vs Predicted ({best_name})")
ax.set_xlabel("actual consume")
ax.set_ylabel("predicted consume")
fig.tight_layout()
fig.savefig(fig_dir / "best_prediction_scatter.png", dpi=220)
plt.close(fig)

ordered = prediction_df.reset_index(drop=True).head(80)
fig, ax = plt.subplots(figsize=(8.0, 4.2))
ax.plot(ordered.index, ordered["actual"], label="actual",
    ↪ color="#111827", lw=1.6)
ax.plot(
    ordered.index,
    ordered["predicted"],
    label="predicted",
    color="#2563EB",
    lw=1.4,
)
ax.set_title(f"First 80 Test Samples ({best_name})")
ax.set_xlabel("test sample index")
ax.set_ylabel("consume (L/100km)")
ax.legend()
fig.tight_layout()
fig.savefig(fig_dir / "best_prediction_curve.png", dpi=220)
plt.close(fig)

if "RandomForest" in best_name:
    fitted_preprocessor = best_model.named_steps["preprocess"]

```

```

feature_names = get_feature_names(fitted_preprocessor)
importances = best_model.named_steps["model"].feature_importances_
importance_df = (
    pd.DataFrame({"feature": feature_names, "importance":
        ↪ importances})
    .sort_values("importance", ascending=False)
    .head(20)
)
else:
    rf_model = models["Full_RandomForest"]
    rf_model.fit(X_train, y_train)
    fitted_preprocessor = rf_model.named_steps["preprocess"]
    feature_names = get_feature_names(fitted_preprocessor)
    importances = rf_model.named_steps["model"].feature_importances_
    importance_df = (
        pd.DataFrame({"feature": feature_names, "importance":
            ↪ importances})
        .sort_values("importance", ascending=False)
        .head(20)
    )
importance_df.to_csv(
    out_dir / "feature_importance_top20.csv", index=False,
    ↪ encoding="utf-8-sig"
)

fig, ax = plt.subplots(figsize=(8.0, 5.2))
plot_df = importance_df.sort_values("importance")
ax.barh(plot_df["feature"], plot_df["importance"], color="#0F766E")
ax.set_title("Top 20 Feature Importances")
ax.set_xlabel("importance")
fig.tight_layout()
fig.savefig(fig_dir / "feature_importance_top20.png", dpi=220)
plt.close(fig)

summary = {
    "best_model": best_name,
    "best_metrics": result_df.iloc[0].to_dict(),
    "train_size": int(len(X_train)),
    "test_size": int(len(X_test)),
}
(out_dir / "summary.json").write_text(

```

```
        json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    return summary

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--data", default="measurements.xlsx")
    parser.add_argument("--output", default="results_fuel_consumption")
    args = parser.parse_args()

    base_dir = Path(__file__).resolve().parent
    data_path = Path(args.data)
    if not data_path.is_absolute():
        data_path = base_dir / data_path

    output_dir = ensure_dir(base_dir / args.output)
    fig_dir = ensure_dir(base_dir / "figures")
    overview_dir = ensure_dir(output_dir / "01_data_overview")
    model_dir = ensure_dir(output_dir / "02_modeling")

    df = load_data(data_path)
    df_engineered = add_features(df)

    save_overview(df, overview_dir)
    plot_eda(df, fig_dir)

    y = df_engineered[TARGET_COL]
    X = df_engineered.drop(columns=[TARGET_COL])
    summary = evaluate_models(X, y, model_dir, fig_dir)

    print(" 数据规模: ", df.shape)
    print(" 缺失值数量: ")
    print(df.isna().sum())
    print("\n 按汽油类型统计油耗: ")
    print(
        df.groupby("gas_type")[TARGET_COL]
        .agg(["count", "mean", "median", "std", "min", "max"])
        .round(4)
    )
    print("\n 最佳模型: ", summary["best_model"])
```



```
print(" 最佳模型测试集指标: ")
for key, value in summary["best_metrics"].items():
    print(f"{key}: {value}")
print(f"\n 结果已保存到: {output_dir}")

if __name__ == "__main__":
    main()
```

关键运行结果汇总

数据规模: (388, 12)

目标变量: consume, 单位 L/100km

训练集样本数: 310

测试集样本数: 78

按汽油类型统计油耗:

E10: 样本数 =160, 均值 =4.9313, 中位数 =4.8000, 标准差 =0.9010

SP98: 样本数 =228, 均值 =4.8991, 中位数 =4.7000, 标准差 =1.1184

测试集模型结果:

Full_RandomForest RMSE=0.6567, MAE=0.4285, R2=0.5247, NRMSE=0.1042

Full_GradientBoosting RMSE=0.8041, MAE=0.4770, R2=0.2874, NRMSE=0.1276

Full_LinearRegression RMSE=0.8513, MAE=0.6271, R2=0.2012, NRMSE=0.1351

Full_Ridge RMSE=0.8558, MAE=0.6302, R2=0.1928, NRMSE=0.1358

GasType_LinearRegression RMSE=0.9514, MAE=0.6396, R2=0.0023, NRMSE=0.1510

随机森林重要特征:

distance=0.5014, distance_per_speed=0.2071, speed_temp_interaction=0.0928,
speed=0.0790, temp_outside=0.0554, temp_diff_inside_outside=0.0239。